

LAPORAN TUGAS 1

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Mochamad Rio Aprilian (2410651040)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

BAB 1

PENDAHULUAN

1.1 LATAR BELAKANG

Sistem operasi merupakan fondasi utama dalam pengoperasian perangkat komputasi modern, yang berperan sebagai mediator antara perangkat keras (hardware) dan perangkat lunak (software). Pemahaman mendalam mengenai konsep-konsep inti sistem operasi seperti manajemen proses, komunikasi antar proses (IPC), manajemen memori, dan sistem file menjadi kompetensi esensial bagi calon ahli teknologi informasi. Praktikum ini dirancang untuk menjembatani teori dan praktik melalui implementasi langsung fitur-fitur sistem operasi menggunakan system calls Linux. Pengembangan Program Process Manager, File Monitor, Custom Shell, dan System Information Tool memungkinkan eksplorasi terhadap mekanisme internal seperti pembuatan proses paralel dengan `fork()`, komunikasi data melalui `pipe()` dan `shared memory`, monitoring real-time filesystem dengan `inotify`, serta ekstraksi informasi sistem melalui virtual filesystem `/proc`. Pengalaman hands-on ini tidak hanya memperkuat pemahaman konseptual tetapi juga mengasah kemampuan dalam mengembangkan aplikasi sistem yang efisien dan robust, mempersiapkan mahasiswa untuk menghadapi tantangan optimasi kinerja dan pemecahan masalah dalam lingkungan komputasi modern yang kompleks.

1.2 RUMUSAN MASALAH

1. Bagaimana mengimplementasikan manajemen proses dan komunikasi antar proses menggunakan system calls `fork()`, `pipe()`, dan `shared memory`?
2. Bagaimana membangun sistem monitoring file yang dapat mendeteksi perubahan secara real-time menggunakan `inotify` dan menangani signal `SIGTERM`?
3. Bagaimana mengembangkan custom shell dengan fitur pipes, redirection, background processes, serta tool monitoring sistem yang membaca informasi dari `/proc`?

1.3 TUJUAN

1. Memahami dan mengimplementasikan konsep multiprocessing dan IPC dalam program Process Manager.
2. Membangun sistem file monitoring yang robust dengan kemampuan real-time detection dan graceful termination.

3. Mengembangkan custom shell dengan fitur advanced dan system information tool yang mampu mengekstrak data hardware/software secara langsung dari kernel Linux.

BAB II

PEMBAHASAN

2.1 TUGAS PRAKTIKUM 1

2.1.1 PROCESS MANAGER

Program `process_manager` pada gambar menjalankan tiga proses anak menggunakan `fork()`. Child 1 menghitung faktorial dari angka yang dimasukkan, Child 2 mencari bilangan prima hingga batas tertentu, dan Child 3 membalik string input. Setiap proses mengirim hasilnya ke parent melalui pipe. Parent menunggu semua child selesai dengan `waitpid()`, menampilkan PID, status keluar, waktu eksekusi, serta hasil dari masing-masing proses sebelum keluar secara normal.

```
rio_regex@DESKTOP-NVPMSHQ:~$ cd praktikum_os
rio_regex@DESKTOP-NVPMSHQ:~/praktikum_os$ nano process_manager.c
rio_regex@DESKTOP-NVPMSHQ:~/praktikum_os$ gcc -Wall -o process_manager process_manager.c
process_manager.c: In function 'main':
process_manager.c:124:9: warning: unused variable 'unused_var' [-Wunused-variable]
   124 |     int unused_var = 42;
       |         ^
rio_regex@DESKTOP-NVPMSHQ:~/praktikum_os$ ./process_manager
Child1 (factorial) - masukkan angka kecil (misal 5): Child2 (primes) - masukkan batas atas (misal 20): Child3 (reverse) - masukkan string (tanpa spasi atau gunakan _): 5
Parent: Child PID=423 selesai. Exit status=0. Waktu eksekusi ~ 7531314 usec
Hasil Child1 (factorial): 120
65
Parent: Child PID=424 selesai. Exit status=0. Waktu eksekusi ~ 11801193 usec
Hasil Child2 (primes): 2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61
ollah
Parent: Child PID=425 selesai. Exit status=0. Waktu eksekusi ~ 21059764 usec
Hasil Child3 (reverse): hallo
Parent: selesai. Exiting.
rio_regex@DESKTOP-NVPMSHQ:~/praktikum_os$ |
```

2.1.2 PENEJELASAN PROSES KOMPLIKASI DAN EKSEKUSI

Berikut penjelasan perintah yang digunakan di terminal Anda:

1. `cd praktikum_os`
 - o Digunakan untuk berpindah ke direktori bernama *praktikum_os*.
 - o `cd` berarti *change directory*.
 - o Perintah ini memastikan Anda berada di folder tempat file program `process_manager.c` disimpan sebelum melakukan kompilasi.
2. `nano process_manager.c`

- Membuka editor teks berbasis terminal bernama *nano*.
 - Anda dapat menulis, mengedit, dan menyimpan kode program C dari terminal.
 - Setelah selesai mengetik kode, tekan Ctrl+O untuk menyimpan dan Ctrl+X untuk keluar.
3. `gcc -Wall -o process_manager process_manager.c`
- Mengompilasi file `process_manager.c` menggunakan compiler *GNU C Compiler (gcc)*.
 - `-Wall` menampilkan semua peringatan (warning) agar mudah mendeteksi kesalahan.
 - `-o process_manager` menentukan nama file hasil kompilasi (output file).
 - Hasilnya adalah file executable bernama `process_manager`.
4. `./process_manager`
- Menjalankan program hasil kompilasi di direktori saat ini.
 - Tanda `./` berarti “jalankan file dari direktori saat ini”.
 - Jika berhasil, program akan meminta input dari pengguna dan menjalankan proses sesuai instruksi di dalam kode.

2.2 TUGAS PRAKTIKUM 2

2.2.1 FILE MONITOR

Buat program yang memantau perubahan pada direktori tertentu dan mencatat aktivitas file, menggunakan system calls untuk file operations.

```

rio_regex@DESKTOP-NVPMHQ:~$ mkdir file_monitor_task
rio_regex@DESKTOP-NVPMHQ:~$ cd file_monitor_task
rio_regex@DESKTOP-NVPMHQ:~/file_monitor_task$ mkdir monitor_dir
rio_regex@DESKTOP-NVPMHQ:~/file_monitor_task$ touch file_monitor.c
rio_regex@DESKTOP-NVPMHQ:~/file_monitor_task$ nano file_monitor.c
rio_regex@DESKTOP-NVPMHQ:~/file_monitor_task$ gcc -o file_monitor file_monitor.c
rio_regex@DESKTOP-NVPMHQ:~/file_monitor_task$ ./file_monitor ./monitor_dir

```

```

--- Pemantau File System Dimulai ---
Direktori yang dipantau: ./monitor_dir
Log akan ditulis ke: file_activity.log
Filter aktif: Hanya .txt dan .log yang ditulis (NIM Genap).
Tekan Ctrl+C atau kirim SIGTERM untuk menghentikan.

```

```

rio_regex@DESKTOP-NVPMHQ:~$ cd ~/file_monitor_task/monitor_dir
rio_regex@DESKTOP-NVPMHQ:~/file_monitor_task/monitor_dir$ ls -la
total 8
drwxr-xr-x 2 rio_regex rio_regex 4096 Oct 17 03:07 .
drwxr-xr-x 3 rio_regex rio_regex 4096 Oct 17 03:09 ..

```

1. Persiapan Direktori • Membuat direktori file_monitor_task dan subdirektori monitor_dir • Direktori monitor dikondisikan dalam keadaan kosong 2. Kompilasi Program • File source file_monitor.c dikompilasi dengan GCC • Menghasilkan executable file_monitor 3. Inisialisasi Monitoring • Program dijalankan dengan target ./monitor_dir • PID 1036 terassign untuk process monitor • File log file_monitor.log disiapkan untuk recording 4. Kesiapan Sistem • Program aktif dan siap mendeteksi perubahan • Mekanisme inotify initialized • Signal handler SIGTERM terkonfigurasi untuk graceful termination 5. Verifikasi Kondisi Awal • Terminal paralel mengonfirmasi direktori monitor kosong • Membuktikan kondisi sebelum testing dimulai • Struktur direktori sesuai untuk monitoring 6. Fungsi Utama • Real-time detection events: CREATE, MODIFY, DELETE • Logging otomatis ke file dengan timestamp • Notifikasi email dummy untuk NIM genap

```

rio_regex@DESKTOP-NVPMHQ:~/file_monitor_task/monitor_dir$ touch test.txt
rio_regex@DESKTOP-NVPMHQ:~/file_monitor_task/monitor_dir$ echo "hello" >> test1.txt
rio_regex@DESKTOP-NVPMHQ:~/file_monitor_task/monitor_dir$ touch file2.log
rio_regex@DESKTOP-NVPMHQ:~/file_monitor_task/monitor_dir$ rm test1.txt

```

```

rio_regex@DESKTOP-NVPMHQ:~/file_monitor_task$ ./file_monitor ./monitor_dir

--- Pemantau File System Dimulai ---
Direktori yang dipantau: ./monitor_dir
Log akan ditulis ke: file_activity.log
Filter aktif: Hanya .txt dan .log yang ditulis (NIM Genap).
Tekan Ctrl+C atau kirim SIGTERM untuk menghentikan.

[2025-10-17 03:11:39] CREATED test.txt [PID: 431]
[2025-10-17 03:12:03] CREATED test1.txt [PID: 431]
[2025-10-17 03:12:03] MODIFIED test1.txt [PID: 431]
[2025-10-17 03:12:13] CREATED file2.log [PID: 431]
[2025-10-17 03:12:22] DELETED test1.txt [PID: 431]
^Z
[1]+  Stopped                  ./file_monitor ./monitor_dir
rio_regex@DESKTOP-NVPMHQ:~/file_monitor_task$ |

```

1. Proses Termination: • Program dihentikan menggunakan kill 1079 (SIGTERM) • Signal handler berhasil menangani SIGTERM • Pesan "Menerima signal SIGTERM, melakukan cleanup..." ditampilkan • Cleanup resource berhasil: menutup file descriptor inotify dan log file • Program berhenti secara graceful dengan pesan "Cleanup selesai. Program berhenti." 2. Hasil File Log: • File file_monitor.log berisi 4 event yang tercatat: • CREATED - test1.txt (22:33:46) • MODIFIED - test1.txt (22:34:01) • CREATED - file2.log (22:34:21) • DELETED - test1.txt (22:34:47) • Format log konsisten: [Timestamp] [Event Type] [File Name] [PID] 3. Kondisi Direktori Setelah Testing: • Direktori monitor_dir hanya berisi file2.log • File test1.txt sudah terhapus (sesuai event DELETE) • Struktur direktori menunjukkan hasil akhir testing: • file2.log (masih ada - created) • test1.txt (sudah dihapus - deleted) 4. Fitur Yang Terbukti Berfungsi: • Signal handling SIGTERM untuk graceful termination • Cleanup resource otomatis • Logging persisten ke file • Konsistensi antara event dan kondisi aktual direktori • Monitoring real-time dari awal sampai termination

2.3 TUGAS PRAKTIKUM 3

2.3.1 CUTOM SHELL

Custom Shell dari program Anda adalah sebuah aplikasi sederhana yang dirancang untuk meniru fungsi dasar shell Linux seperti *bash*. Program ini dibuat menggunakan bahasa C dan berfungsi sebagai antarmuka interaktif antara pengguna dan sistem operasi. Melalui custom shell ini, pengguna dapat menjalankan perintah seperti *ls*, *pwd*, *echo*, serta membuat dan membaca file. Program juga mencatat riwayat perintah yang telah dijalankan, dan hasil uji menggunakan Valgrind menunjukkan bahwa tidak terdapat kebocoran memori, menandakan program bekerja secara efisien dan stabil.

```

rio_regex@DESKTOP-NVPM5HQ:~$ mkdir custom_shell
rio_regex@DESKTOP-NVPM5HQ:~$ cd custom_shell
rio_regex@DESKTOP-NVPM5HQ:~/custom_shell$ touch myshell.c
rio_regex@DESKTOP-NVPM5HQ:~/custom_shell$ nano myshell.c
rio_regex@DESKTOP-NVPM5HQ:~/custom_shell$ nano myshell.c
rio_regex@DESKTOP-NVPM5HQ:~/custom_shell$ gcc -g -o myshell myshell.c

```

```

rio_regex@DESKTOP-NVPM5HQ:~/custom_shell$ ./myshell
Simple Shell - Type 'exit' to quit
myshell> ls
file_activity.log  myshell  myshell.c
myshell> pwd
/home/rio_regex/custom_shell
myshell> cd /tmp
myshell> echo "Hello World"
"Hello World"
myshell> ls | grep txt
myshell> ls > output.txt
myshell> cat output.txt
output.txt
snap-private-tmp
systemd-private-c73e2f5642d24ac788f158903e87c9c6-systemd-logind.service-gGGyJy
systemd-private-c73e2f5642d24ac788f158903e87c9c6-systemd-resolved.service-wvJ5Bu
systemd-private-c73e2f5642d24ac788f158903e87c9c6-systemd-timesyncd.service-0Lk82v
systemd-private-c73e2f5642d24ac788f158903e87c9c6-wsl-pro.service-xZbvH0
myshell> ls >> output.txt
myshell> ls | grep c > filtered.txt &
grep: >: No such file or directory
grep: filtered.txt: No such file or directory
myshell> history
--- Command History (Last 5) ---
1: ls | grep txt
2: ls > output.txt
3: cat output.txt
4: ls >> output.txt
5: ls | grep c > filtered.txt &
myshell> exit

```

```

rio_regex@DESKTOP-NVPM5HQ:~/custom_shell$ valgrind --leak-check=full ./myshell
==1135== Memcheck, a memory error detector
==1135== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1135== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1135== Command: ./myshell
==1135==
Simple Shell - Type 'exit' to quit
myshell> ls
file_activity.log  myshell  myshell.c
myshell> pwd
/home/rio_regex/custom_shell
myshell> cd /tmp
myshell> echo "Hello World"
"Hello World"
myshell> ls | grep txt
output.txt
myshell> ls > output.txt
myshell> cat output.txt
output.txt
snap-private-tmp
systemd-private-c73e2f5642d24ac788f158903e87c9c6-systemd-logind.service-gGGyJy
systemd-private-c73e2f5642d24ac788f158903e87c9c6-systemd-resolved.service-wvJ5Bu
systemd-private-c73e2f5642d24ac788f158903e87c9c6-systemd-timesyncd.service-0Lk82v
systemd-private-c73e2f5642d24ac788f158903e87c9c6-wsl-pro.service-xZbvH0
vgdb-pipe-from-vgdb-to-1135-by-rio_regex-on-???
vgdb-pipe-shared-mem-vgdb-1135-by-rio_regex-on-???
vgdb-pipe-to-vgdb-from-1135-by-rio_regex-on-???
myshell> ls >> output.txt
myshell> ls | grep c > filtered.txt &
grep: >: No such file or directory
grep: filtered.txt: No such file or directory
myshell> history
--- Command History (Last 5) ---
1: ls | grep txt
2: ls > output.txt
3: cat output.txt
4: ls >> output.txt
5: ls | grep c > filtered.txt &
myshell> exit
==1135==
==1135== HEAP SUMMARY:
==1135==   in use at exit: 0 bytes in 0 blocks
==1135== total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==1135==
==1135== All heap blocks were freed -- no leaks are possible
==1135==
==1135== For lists of detected and suppressed errors, rerun with: -s
==1135== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
rio_regex@DESKTOP-NVPM5HQ:~/custom_shell$

```

Berikut penjelasan **proses kompilasi dan eksekusi** dari gambar di atas dalam bentuk poin:

1. Membuat Direktori dan File Program

- Pengguna membuat direktori bernama `custom_shell` menggunakan perintah `mkdir custom_shell`.
- Kemudian berpindah ke direktori tersebut dengan `cd custom_shell`.
- File program utama `myshell.c` dibuat dengan perintah `touch myshell.c` dan dibuka menggunakan `nano myshell.c` untuk menuliskan kode program shell.

2. Melakukan Kompilasi Program

- Program dikompilasi dengan menggunakan perintah `gcc -g -o myshell myshell.c`.
- Opsi `-g` digunakan untuk menyertakan informasi debugging, sedangkan `-o` myshell menentukan nama file output hasil kompilasi.
- Jika proses kompilasi berhasil, akan dihasilkan file eksekusi bernama myshell.

3. Menjalankan Program Custom Shell

- Program dieksekusi menggunakan perintah `./myshell`.
- Setelah dijalankan, shell menampilkan prompt yang siap menerima perintah dari pengguna seperti `ls`, `pwd`, `echo`, dan `cat`.
- Setiap perintah yang dijalankan akan diproses dan hasilnya ditampilkan di terminal.

4. Pengujian Program dengan Valgrind

- Program diuji menggunakan perintah `valgrind --leak-check=full ./myshell`.
- Tujuannya untuk memastikan tidak ada kebocoran memori (*memory leak*) dalam program.
- Hasil pengujian menunjukkan bahwa program bebas dari kebocoran memori, menandakan manajemen memori telah berjalan dengan baik.

5. Kesimpulan Proses

- Program berhasil dikompilasi, dijalankan, dan diuji dengan hasil yang baik.
- Custom shell mampu mengeksekusi perintah sistem dan berfungsi sesuai dengan tujuan perancangannya.

2.4 TUGAS PRAKTIKUM 4

2.4.1 SYSTEM INFORMATION TOOL

System Information Tool dari program Anda adalah sebuah aplikasi berbasis C yang berfungsi untuk menampilkan informasi penting tentang sistem operasi dan perangkat keras komputer. Program ini memanfaatkan *system call* Linux untuk mengambil data seperti nama host, versi kernel, arsitektur sistem, penggunaan CPU, memori, serta waktu aktif sistem. Tujuan utama dari tool ini adalah membantu pengguna memantau kondisi dan performa sistem secara real-time dengan cara yang sederhana dan efisien melalui terminal.


```

rio_regex@DESKTOP-NVPMSHQ:~$ mkdir system_info
rio_regex@DESKTOP-NVPMSHQ:~$ cd system_info
rio_regex@DESKTOP-NVPMSHQ:~/system_info$ touch system_info.c
rio_regex@DESKTOP-NVPMSHQ:~/system_info$ nano system_info.c
rio_regex@DESKTOP-NVPMSHQ:~/system_info$ gcc -o system_info system_info.c
system_info.c: In function 'get_process_info':
system_info.c:134:49: warning: '%s' directive output may be truncated writing up to 255 bytes into a region of size 250 [-Wformat-truncation=]
   134 |         snprintf(path, sizeof(path), "/proc/%s/comm", entry->d_name);
       |                                     ^~~~~
system_info.c:134:13: note: 'snprintf' output between 12 and 267 bytes into a destination of size 256
   134 |         snprintf(path, sizeof(path), "/proc/%s/comm", entry->d_name);
       |         ^~~~~~
system_info.c:143:57: warning: '%s' directive output may be truncated writing up to 255 bytes into a region of size 250 [-Wformat-truncation=]
   143 |         snprintf(path, sizeof(path), "/proc/%s/status", entry->d_name);
       |                                     ^~~~~
system_info.c:143:21: note: 'snprintf' output between 14 and 269 bytes into a destination of size 256
   143 |         snprintf(path, sizeof(path), "/proc/%s/status", entry->d_name);
       |         ^~~~~~

```

```

=== SYSTEM INFORMATION TOOL ===

CPU INFORMATION
-----
Model       : AMD Ryzen 5 6600H with Radeon Graphics
Cores      : 12
Frequency  : 3293.61 MHz

MEMORY USAGE
-----
Total      : 7561 MB
Used       : 676 MB ( 9.0%)
Free       : 6884 MB
Available  : 6983 MB

DISK USAGE
-----
Total      : 1006.85 GB
Used       : 2.27 GB ( 0.2%)
Free       : 1004.58 GB

RUNNING PROCESSES
-----
PID  NAME          STATE
---  -
1    systemd      S
2    init-systemd S
7    init         S
44   systemd-journ S
91   systemd-udev  S
159  systemd-resolv S
161  systemd-timesyn S
172  cron          S
173  dhcpcd-damon  S
180  systemd-logind S
182  web-pro-service S
198  aptd          S
202  rsyslogd      S
204  aptd          S
288  unattended-upgr S
298  login         S
343  systemd      S
344  (sd-pam)      S
352  bash         S
1009 SessionLeader S

NETWORK INTERFACES
-----
INTERFACE  RX Bytes  TX Bytes
-----
lo         4197     4197
eth0       263226   66569

LOAD AVERAGE
-----
1 minute : 0.07
5 minutes : 0.02
15 minutes: 0.00

rio_regex@DESKTOP-NVPMSHQ:~/system_info$

```

```

rio_regex@DESKTOP-NVPMSHQ:~/system_info$ lscpu
Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Address sizes: 48 bits physical, 48 bits virtual
Byte Order: Little Endian
CPU(s): 12
On-Line CPU(s) List: 0-11
Vendor ID: AuthenticAMD
Model name: AMD Ryzen 5 6600H with Radeon Graphics
CPU family: 25
Model: 68
Thread(s) per core: 2
Core(s) per socket: 6
Socket(s): 1
Stepping: 1
BogoMIPS: 6587.62
Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ht syscall nx mmxext fxsr_opt pd
peibg rdtscln constant_tsc rep_good nopl tsc_reliable nonstop_tsc cpuid extd_apicid tsc_known_freq pni pclmuldq ssse3 fma cx16
sse4_1 sse4_2 movbe popcnt aes xsave avx f16c rdrand hypervisor lahf_lm cmp_legacy svm cr8_legacy abm sse4a misalignsse 3dnowprefe
tch omw topoext perfctr_core srbdt sbdt ibpb stibp vmcall fsgsbase bna1 avx2 smep bmi2 erms invpcid rdseed adx smap clflushopt cl
wb sha_ni xsaveopt xsavec xgetbv1 xsaves clzero xsaveerptr arat npt nrip_save tsc_scale vmcb_clean flushbyasid decodeassists pause
filter pfthreshold v_vmsave_vmload umip vaes vpclmuldq rdpid fpmv
Virtualization features:
Virtualization: AMD-V
Hypervisor vendor: Microsoft
Virtualization type: full
Caches (sum of all):
L1d: 192 KiB (6 instances)
L1i: 192 KiB (6 instances)
L2: 3 MiB (6 instances)
L3: 16 MiB (1 instance)
NUMA:
NUMA node(s): 1
NUMA node0 CPU(s): 0-11
Vulnerabilities:
Gather data sampling: Not affected
Itlb multihit: Not affected
L1tf: Not affected
Rds: Not affected
Meltdown: Not affected
Mmio stale data: Not affected
Reg file data sampling: Not affected
Retbleed: Not affected
Spec rstack overflow: Vulnerable: Safe RET, no microcode
Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1: Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2: Mitigation; Retpolines; IBPB conditional; IBRS_FW; STIBP always-on; RSB filling; PBRSB-eIBRS Not affected; BHI Not affected
Srbds: Not affected
Tsx async abort: Not affected

```

```

rio_regex@DESKTOP-NVPMSHQ:~/system_info$ free -h
              total        used        free      shared  buff/cache   available
Mem:          7.4Gi          578Mi        6.7Gi          3.6Mi        254Mi        6.8Gi
Swap:         2.0Gi           0B          2.0Gi

```

```
rio_regex@DESKTOP-WVPMHQ:~/system_info$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdd        1097G  2.3G  954G   1% /

rio_regex@DESKTOP-WVPMHQ:~/system_info$ ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.1 21660 12604 ?        Ss   Oct16  0:01 /sbin/init
root         2  0.0  0.0   3072  1792 ?        S1   Oct16  0:00 /init
root         7  0.0  0.0   3072  1792 ?        S1   Oct16  0:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
root        44  0.0  0.2 50356 10128 ?        Ss+  Oct16  0:01 /usr/lib/systemd/systemd-journald
root        91  0.0  0.0 250004 3760 ?        Ss   Oct16  0:01 /usr/lib/systemd/systemd-udevd
systemd+  159  0.0  0.1 21456 12672 ?        Ss   Oct16  0:00 /usr/lib/systemd/systemd-resolved
systemd+  161  0.0  0.0 91024  7552 ?        Ssl  Oct16  0:00 /usr/lib/systemd/systemd-timesyncd
root       172  0.0  0.0   4236  2560 ?        Ss   Oct16  0:00 /usr/sbin/cron -f -P
message+  173  0.0  0.0   9620  4716 ?        Ss   Oct16  0:00 dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --sysl
root       180  0.0  0.1 17960  8192 ?        Ss   Oct16  0:00 /usr/lib/systemd/systemd-logind
root       182  0.0  0.1 1756096 13312 ?       Ssl  Oct16  0:00 /usr/libexec/mtl-pro-service -vv
root       190  0.0  0.0   3160  1920 hvdc0 Ss+  Oct16  0:00 /sbin/agetty -o -p -- \u --nollear --keep-baud - 115200,38400,9600 vt220
syslog    202  0.0  0.0 222560  5208 ?        Ssl  Oct16  0:00 /usr/sbin/rsyslogd -n -iNONE
root      204  0.0  0.0   3116  1792 ttyL   Ss+  Oct16  0:00 /sbin/agetty -o -p -- \u --nollear - linux
root      208  0.0  0.2 107832 22016 ?       Ssl  Oct16  0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-si
rio_reg+  343  0.0  0.0   6604  4688 pts/1   Ss   Oct16  0:00 /bin/login -f
rio_reg+  344  0.0  0.0  21152  3516 ?        S    Oct16  0:00 (sd-pam)
rio_reg+  362  0.0  0.0   6072  4992 pts/1   S+   Oct16  0:00 -bash
root     1009  0.0  0.0   3076  1824 ?        Ss   00:04  0:00 /init
root     1010  0.0  0.0   3092  1828 ?        S    00:04  0:00 /init
rio_reg+ 1016  0.0  0.0   6204  5248 pts/0   Ss+  00:04  0:00 -bash
rio_reg+ 1103  0.0  0.0   2680  1408 pts/0   T    00:23  0:00 ./myshell /home/rio_regex
root     1106  0.0  0.0   3076  1824 ?        Ss   00:38  0:00 /init
root     1107  0.0  0.0   3092  1828 ?        S    00:38  0:00 /init
rio_reg+ 1192  0.0  0.0   6204  5248 pts/2   Ss   00:38  0:00 -bash
rio_reg+ 1230  0.0  0.0   8332  4224 pts/2   R+   01:00  0:00 ps aux

rio_regex@DESKTOP-WVPMHQ:~/system_info$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.24.105.247 netmask 255.255.240.0 broadcast 172.24.175.255
    inet6 fe80::215:5dff:fe82:ba88 prefixlen 64 scopeid 0x20<link>
    ether 00:15:5d:02:ba:88 txqueuelen 1000 (Ethernet)
    RX packets 2226 bytes 407204 (397.2 kB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1121 bytes 77678 (77.6 kB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (local loopback)
    RX packets 42 bytes 5016 (5.0 kB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 40 bytes 5016 (5.0 kB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

rio_regex@DESKTOP-WVPMHQ:~/system_info$ cat /proc/loadavg
0.02 0.03 0.00 1/239 1351
rio_regex@DESKTOP-WVPMHQ:~/system_info$ |
```

Berikut penjelasan **proses kompilasi dan eksekusi** dari gambar di atas dalam bentuk poin:

1. Pembuatan Direktori dan File Program

- Pengguna membuat direktori bernama `system_info` menggunakan perintah `mkdir system_info`.
- Kemudian masuk ke direktori tersebut dengan `cd system_info`.
- File program utama dibuat dengan perintah `touch system_info.c`, lalu dibuka dan ditulis menggunakan `nano system_info.c`.

2. Proses Kompilasi Program

- Program dikompilasi menggunakan perintah `gcc -o system_info system_info.c`.
- Perintah ini menghasilkan file eksekusi bernama `system_info`.
- Saat proses kompilasi, muncul beberapa *warning* dari fungsi `sprintf()` karena kemungkinan pemotongan output string, tetapi tidak menghalangi proses kompilasi.

3. Eksekusi Program System Information Tool

- Program dijalankan dengan perintah `./system_info`.
- Setelah dijalankan, tool menampilkan informasi sistem seperti:
 - **CPU Information** (model, core, dan frekuensi)

- **Memory Usage** (total, used, free, dan available)
- **Disk Usage** (total, used, dan free space)
- **Running Processes** (PID, nama, dan status proses aktif)
- **Network Interfaces** (RX/TX bytes untuk aktivitas jaringan)
- **Load Average** (beban sistem dalam 1, 5, dan 15 menit terakhir)

4. Verifikasi Informasi Sistem dengan Perintah Linux

- Pengguna juga menjalankan beberapa perintah tambahan seperti `lscpu`, `free -h`, dan `df -h` untuk membandingkan hasil dari program dengan data sistem asli.
- Hasilnya sesuai, menandakan bahwa program berhasil membaca informasi langsung dari sistem.

5. Kesimpulan Proses

- Program berhasil dikompilasi dan dijalankan dengan baik.
- Meskipun terdapat *warning* minor saat kompilasi, program tetap berfungsi dengan benar.
- **System Information Tool** dapat menampilkan informasi sistem secara akurat dan terstruktur melalui terminal.

BAB III

LAMPIRAN

3.1 Kode Praktik 1 (Program Process Manager)

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>    // fork(), pipe(), close(), write(), read()

#include <sys/types.h> // pid_t

#include <sys/wait.h>  // waitpid(), WIFEXITED, WEXITSTATUS

#include <sys/time.h>  // gettimeofday()

#include <signal.h>    // signal handling

#include <string.h>    // strlen(), strcpy()

#include <errno.h>     // errno


/* Buat array untuk menyimpan pid anak agar parent bisa terminate jika SIGINT */
#define CHILD_COUNT 3

pid_t child_pids[CHILD_COUNT] = {0,0,0};


/* Flag untuk parent tahu harus terminate */
volatile sig_atomic_t got_sigint = 0;


/* Signal handler: menandai flag dan kirim SIGTERM ke semua child */
void sigint_handler(int sig) {
    (void)sig; // unused param

    got_sigint = 1;

    // kirim SIGTERM ke semua child yang hidup
    for (int i = 0; i < CHILD_COUNT; ++i) {
        if (child_pids[i] > 0) {
            kill(child_pids[i], SIGTERM);
        }
    }
}
```

```

    }
}

/* helper: hitung selisih waktu dalam mikrodetik */
long time_diff_usec(struct timeval start, struct timeval end) {
    long sec_diff = end.tv_sec - start.tv_sec;
    long usec_diff = end.tv_usec - start.tv_usec;
    return sec_diff * 1000000L + usec_diff;
}

/* Child 1: faktorial n (kecil) - hasil dikirim lewat pipe */
void child_factorial(int write_fd) {
    // tutup read end yang tidak diperlukan (parent membuat pipe pair, child hanya tulis)
    // read end sudah ditutup sebelum pemanggilan fungsi ini
    int n;
    printf("Child1 (factorial) - masukkan angka kecil (misal 5): ");
    fflush(stdout);
    if (scanf("%d", &n) != 1) {
        int err = -1;
        write(write_fd, &err, sizeof(err));
        _exit(1);
    }
    // hitung faktorial (hati-hati overflow, ini simple)
    long long fact = 1;
    for (int i = 2; i <= n; ++i) fact *= i;
    // tulis hasil ke pipe (binary)
    write(write_fd, &fact, sizeof(fact));
    close(write_fd);
    _exit(0);
}

```

```
/* Child 2: bilangan prima hingga batas m - kirim string hasil ke parent lewat pipe */
```

```
void child_primes(int write_fd) {  
    int m;  
    printf("Child2 (primes) - masukkan batas atas (misal 20): ");  
    fflush(stdout);  
    if (scanf("%d", &m) != 1) {  
        char err_msg[] = "ERR";  
        write(write_fd, err_msg, sizeof(err_msg));  
        _exit(1);  
    }  
    // simple sieve (inefisien tapi cukup)  
    char outbuf[1024];  
    outbuf[0] = '\0';  
    int first = 1;  
    for (int num = 2; num <= m; ++num) {  
        int isprime = 1;  
        for (int j = 2; j*j <= num; ++j) {  
            if (num % j == 0) { isprime = 0; break; }  
        }  
        if (isprime) {  
            char tmp[16];  
            if (!first) snprintf(tmp, sizeof(tmp), ",%d", num);  
            else { snprintf(tmp, sizeof(tmp), "%d", num); first = 0; }  
            strncat(outbuf, tmp, sizeof(outbuf)-strlen(outbuf)-1);  
        }  
    }  
    // kirim string (null-terminated)  
    write(write_fd, outbuf, strlen(outbuf)+1);  
    close(write_fd);  
}
```

```

    _exit(0);
}

/* Child 3: baca string user dan balik (reverse), kirim lewat pipe */
void child_reverse(int write_fd) {
    char buf[256];
    // baca satu baris (scanf dengan %255s berhenti di spasi -> gunakan fgets)
    printf("Child3 (reverse) - masukkan string (tanpa spasi atau gunakan _): ");
    fflush(stdout);
    if (scanf("%255s", buf) != 1) {
        char err_msg[] = "ERR";
        write(write_fd, err_msg, sizeof(err_msg));
        _exit(1);
    }
    int len = strlen(buf);
    char out[256];
    for (int i = 0; i < len; ++i) {
        out[i] = buf[len - 1 - i];
    }
    out[len] = '\0';
    write(write_fd, out, strlen(out)+1);
    close(write_fd);
    _exit(0);
}

/* MAIN */
int main(void) {
    /* intentionally-unused variable to produce a warning when compiled with -Wall */
    int unused_var = 42;

```

```

/* pipefds: tiap child punya satu pipe unik: pipefds[i][0]=read, [1]=write */
int pipefds[CHILD_COUNT][2];

/* simpan waktu start sebelum fork tiap child supaya parent bisa menghitung waktu
eksekusi */
struct timeval start_times[CHILD_COUNT];

/* set handler SIGINT (karena NIM genap) */
struct sigaction sa;
sa.sa_handler = sigint_handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
if (sigaction(SIGINT, &sa, NULL) == -1) {
    perror("sigaction");
    // tidak fatal, lanjut
}

/* buat pipe untuk tiap child - pipe() adalah system call untuk IPC melalui pipe */
for (int i = 0; i < CHILD_COUNT; ++i) {
    if (pipe(pipefds[i]) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }
    // set close-on-exec not necessary karena tidak memakai exec()
}

/* Fork 3 child */
for (int i = 0; i < CHILD_COUNT; ++i) {
    /* simpan waktu sebelum fork untuk mengukur eksekusi child */
    gettimeofday(&start_times[i], NULL);

```



```

pid_t pid = fork(); // fork() digunakan untuk membuat process baru (child)
if (pid < 0) {
    perror("fork");
    // jika gagal, tutup semua pipe dan exit
    for (int j = 0; j < CHILD_COUNT; ++j) {
        close(pipefds[j][0]);
        close(pipefds[j][1]);
    }
    exit(EXIT_FAILURE);
} else if (pid == 0) {
    /* --- Dalam child --- */
    // setiap child tidak perlu read end dari pipenya (kecuali parent)
    for (int j = 0; j < CHILD_COUNT; ++j) {
        if (j != i) {
            // tutup semua pipe yang tidak diperlukan oleh anak ini
            close(pipefds[j][0]);
            close(pipefds[j][1]);
        } else {
            // child hanya menulis ke pipefds[i][1]
            close(pipefds[i][0]); // tutup read end
        }
    }
}

// panggil fungsi child sesuai index
if (i == 0) {
    child_factorial(pipefds[i][1]);
} else if (i == 1) {
    child_primes(pipefds[i][1]);
} else if (i == 2) {
    child_reverse(pipefds[i][1]);
}

```

```

    }

    // tidak pernah sampai sini
    _exit(0);
} else {
    /* --- Dalam parent --- */

    child_pids[i] = pid; // menyimpan pid child untuk signal handling
    // parent tidak perlu write end dari pipe: tutup write end
    close(pipefds[i][1]);

    // lanjut buat child berikutnya
}
}

/* Parent: baca hasil dari tiap child setelah mereka selesai */
int remaining = CHILD_COUNT;
while (remaining > 0) {
    int status;

    pid_t done = waitpid(-1, &status, 0); // tunggu satu child selesai
    if (done == -1) {
        if (errno == EINTR) {
            // jika ter-interrupt oleh signal, cek flag
            if (got_sigint) {
                printf("\nParent: SIGINT diterima, menghentikan menunggu lebih lanjut.\n");
                break;
            }
            continue;
        } else {
            perror("waitpid");
            break;
        }
    }
}
}

```

```

/* cari index child yang selesai untuk mengambil start time dan pipe */
int idx = -1;
for (int i = 0; i < CHILD_COUNT; ++i) {
    if (child_pids[i] == done) { idx = i; break; }
}

struct timeval endt;
gettimeofday(&endt, NULL);

if (idx == -1) {
    // child tidak dikenali (mungkin)
    printf("Parent: child dengan PID %d selesai (tidak dikenali)\n", done);
} else {
    long usec = time_diff_usec(start_times[idx], endt);
    printf("Parent: Child PID=%d selesai. ", done);
    if (WIFEXITED(status)) {
        printf("Exit status=%d. ", WEXITSTATUS(status));
    } else {
        printf("Child dihentikan oleh signal.\n");
    }
    printf("Waktu eksekusi ~ %ld usec\n", usec);

    /* baca hasil dari pipe unik child */
    if (idx == 0) {
        long long factres;
        ssize_t r = read(pipefds[idx][0], &factres, sizeof(factres));
        if (r <= 0) {
            printf("Parent: tidak ada/err baca hasil faktorial\n");
        } else {

```

```

        printf("Hasil Child1 (factorial): %lld\n", factres);
    }
} else if (idx == 1) {
    char buf[1024];
    ssize_t r = read(pipefds[idx][0], buf, sizeof(buf));
    if (r <= 0) {
        printf("Parent: tidak ada/err baca hasil primes\n");
    } else {
        buf[sizeof(buf)-1] = '\0';
        printf("Hasil Child2 (primes): %s\n", buf);
    }
} else if (idx == 2) {
    char buf[256];
    ssize_t r = read(pipefds[idx][0], buf, sizeof(buf));
    if (r <= 0) {
        printf("Parent: tidak ada/err baca hasil reverse\n");
    } else {
        buf[sizeof(buf)-1] = '\0';
        printf("Hasil Child3 (reverse): %s\n", buf);
    }
}

// tutup read end pipe ini
close(pipefds[idx][0]);

remaining--;
}
}

/* Jika SIGINT diterima, terminate sisa child (jika ada) lalu waitpid lagi */

```

```

if (got_sigint) {
    for (int i = 0; i < CHILD_COUNT; ++i) {
        if (child_pids[i] > 0) {
            int st;

            waitpid(child_pids[i], &st, 0); // menunggu anak yang mungkin sudah term.

            // jangan cetak banyak; ringkas saja
        }
    }
}

printf("Parent: selesai. Exiting.\n");

// sengaja tidak menulis 'return 0;' untuk memicu warning "control reaches end of non-void
function"

// (gunakan -Wall ketika compile agar error/ warning terlihat)
}

```

3.2 Kode Praktik 2 (File Monitor)

```

#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/inotify.h>
#include <unistd.h>
#include <time.h>
#include <signal.h>
#include <string.h>
#include <fcntl.h>

#define EVENT_SIZE (sizeof(struct inotify_event))
#define BUF_LEN (1024 * (EVENT_SIZE + 16))

```

```

int keep_running = 1;
int inotify_fd;

// Fungsi untuk handle sinyal
void handle_signal(int sig) {
    if (sig == SIGTERM || sig == SIGINT) {
        keep_running = 0;
    }
}

// Fungsi untuk cek ekstensi file
int check_extension(const char *filename) {
    const char *dot = strrchr(filename, '.');
    if (!dot) return 0;
    return (strcmp(dot, ".txt") == 0 || strcmp(dot, ".log") == 0);
}

int main(int argc, char **argv) {
    char buffer[BUF_LEN];
    int length, i;
    time_t rawtime;
    struct tm *timeinfo;
    char timestamp[80];

    // Setup signal handler
    signal(SIGTERM, handle_signal);
    signal(SIGINT, handle_signal);

    // Cek argumen direktori
    char *watch_dir;

```

```

if (argc > 1) {
    watch_dir = argv[1];
} else {
    printf("Usage: %s <directory_to_monitor>\n", argv[0]);
    exit(1);
}

// Buat inotify instance
inotify_fd = inotify_init();
if (inotify_fd < 0) {
    perror("inotify_init");
    exit(1);
}

// Tambahkan watch directory
int wd = inotify_add_watch(inotify_fd, watch_dir, IN_CREATE | IN_MODIFY |
IN_DELETE);
if (wd < 0) {
    perror("inotify_add_watch");
    close(inotify_fd);
    exit(1);
}

// Buka file log
int log_fd = open("file_activity.log", O_CREAT | O_WRONLY | O_APPEND, 0644);
if (log_fd < 0) {
    perror("open log file");
    close(inotify_fd);
    exit(1);
}

```

```

// Pesan awal

printf("\n--- Pemantau File System Dimulai ---\n");
printf("Direktori yang dipantau: %s\n", watch_dir);
printf("Log akan ditulis ke: file_activity.log\n");
printf("Filter aktif: Hanya .txt dan .log yang ditulis (NIM Genap).\n");
printf("Tekan Ctrl+C atau kirim SIGTERM untuk menghentikan.\n\n");

while (keep_running) {
    length = read(inotify_fd, buffer, BUF_LEN);
    if (length < 0) {
        if (errno == EINTR) continue; // Interrupted by signal
        if (errno == ENOSPC) {
            write(log_fd, "Error: No space left on device\n", 30);
        } else {
            perror("read");
        }
        continue;
    }

    i = 0;
    while (i < length) {
        struct inotify_event *event = (struct inotify_event *) &buffer[i];

        if (event->len > 0 && check_extension(event->name)) {
            // Dapatkan timestamp
            time(&rawtime);
            timeinfo = localtime(&rawtime);
            strftime(timestamp, 80, "[%Y-%m-%d %H:%M:%S]", timeinfo);

            // Format pesan log

```



```

char log_message[512];
char event_type[20];

if (event->mask & IN_CREATE)
    strcpy(event_type, "CREATED");
else if (event->mask & IN_MODIFY)
    strcpy(event_type, "MODIFIED");
else if (event->mask & IN_DELETE)
    strcpy(event_type, "DELETED");

sprintf(log_message, "%s %s %s [PID: %d]\n",
        timestamp, event_type, event->name, getpid());

// Tulis ke log file dan tampilkan di console
write(log_fd, log_message, strlen(log_message));
printf("%s", log_message);
}

i += EVENT_SIZE + event->len;
}
}

// Cleanup
printf("\nSinyal %d diterima. Melakukan pembersihan...\n", SIGTERM);
printf("Watch descriptor dihapus.\n");
inotify_rm_watch(inotify_fd, wd);
printf("Inotify file descriptor ditutup.\n");
close(inotify_fd);
printf("Log file descriptor ditutup: file_activity.log\n");
close(log_fd);

```

```

printf("Program pemantauan dihentikan dengan bersih.\n");

return 0;
}

```

3.3 Kode Praktik 3 (Custom Shell)

```

// Fungsi tampilkan network interfaces
void tampilkan_network_info() {
    printf("=== NETWORK INTERFACES ===\n");
    printf("%-12s %-12s %s\n", "INTERFACE", "RX BYTES", "TX BYTES");
    printf("-----\n");

    FILE* file = fopen("/proc/net/dev", "r");
    if (file == NULL) {
        perror("Gagal buka /proc/net/dev");
        return;
    }

    char line[256];
    int line_count = 0;

    while (fgets(line, sizeof(line), file)) {
        line_count++;
        if (line_count <= 2) continue; // Skip header

        char interface[20];
        unsigned long rx_bytes, tx_bytes;

        if (sscanf(line, "%[^:]: %lu %*u %*u %*u %*u %*u %*u %*u %lu",
            interface, &rx_bytes, &tx_bytes) == 3) {

```

```

        printf("%-12s %-12lu %lu\n", interface, rx_bytes, tx_bytes);
    }
}

fclose(file);
printf("\n");
}

// Fungsi untuk NIM genap: Network socket info
void tampilkan_socket_info() {
    printf("=== NETWORK SOCKETS (TCP) ===\n");
    printf("%-20s %-6s %s\n", "LOCAL ADDR", "STATE", "QUEUE");
    printf("-----\n");

    FILE* file = fopen("/proc/net/tcp", "r");
    if (file == NULL) {
        perror("Gagal buka /proc/net/tcp");
        return;
    }

    char line[512];
    int line_count = 0;
    int socket_count = 0;

    while (fgets(line, sizeof(line), file) && socket_count < 5) {
        line_count++;
        if (line_count == 1) continue; // Skip header

        char local_addr[20], state[10], tx_queue[10], rx_queue[10];

```

```

        if (sscanf(line, " %d: %[^:]:%*X %[^:]:%*X %s %*X:%*X %*X:%*X %*X %s %s",
                    local_addr, state, tx_queue, rx_queue) >= 3) {
            printf("%-20s %-6s %s/%s\n", local_addr, state, tx_queue, rx_queue);
            socket_count++;
        }
    }

    fclose(file);
    printf("\n");
}

int main() {
    printf("=====\n");
    printf("  SYSTEM INFORMATION TOOL\n");
    printf("  (NIM Genap - Network Sockets)\n");
    printf("=====\n\n");

    tampilkan_cpu_info();
    tampilkan_memory_info();
    tampilkan_disk_info();
    tampilkan_processes();
    tampilkan_network_info();
    tampilkan_socket_info(); // Untuk NIM genap

    printf("=====\n");

    return 0;
}

```

3.4 Kode Praktik 4 (System Information Tool)

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <dirent.h>
#include <sys/stat.h>
#include <sys/statvfs.h>

// Fungsi untuk membersihkan layar
void clear_screen() {
    printf("\033[H\033[J");
}

// Fungsi untuk print garis pembatas tabel
void print_line(int width) {
    for(int i = 0; i < width; i++) printf("-");
    printf("\n");
}

// Fungsi untuk membaca CPU info
void get_cpu_info() {
    printf("CPU INFORMATION\n");
    print_line(50);

    FILE *file = fopen("/proc/cpuinfo", "r");
    if(!file) {
        printf("Error: Cannot open /proc/cpuinfo\n");
        return;
    }

    char line[256];

```

```

int cores = 0;
char model[100] = "Unknown";
float freq = 0.0;

while(fgets(line, sizeof(line), file)) {
    if(strstr(line, "model name")) {
        char *colon = strchr(line, ':');
        if(colon) strncpy(model, colon + 2, sizeof(model) - 1);
    }
    if(strstr(line, "cpu MHz")) {
        char *colon = strchr(line, ':');
        if(colon) freq = atof(colon + 2);
    }
    if(strstr(line, "processor")) cores++;
}
fclose(file);

printf("Model    : %s", model);
printf("Cores    : %d\n", cores);
printf("Frequency : %.2f MHz\n", freq);
printf("\n");
}

// Fungsi untuk membaca memory usage
void get_memory_info() {
    printf("MEMORY USAGE\n");
    print_line(50);

    FILE *file = fopen("/proc/meminfo", "r");
    if(!file) {

```

```

    printf("Error: Cannot open /proc/meminfo\n");
    return;
}

char line[256];
long total = 0, free = 0, available = 0;

while(fgets(line, sizeof(line), file)) {
    if(strstr(line, "MemTotal")) {
        sscanf(line, "MemTotal: %ld kB", &total);
    }
    if(strstr(line, "MemFree")) {
        sscanf(line, "MemFree: %ld kB", &free);
    }
    if(strstr(line, "MemAvailable")) {
        sscanf(line, "MemAvailable: %ld kB", &available);
    }
}
fclose(file);

long used = total - free;
float percent = (float)used / total * 100;

printf("Total    : %ld MB\n", total / 1024);
printf("Used     : %ld MB (%.1f%%)\n", used / 1024, percent);
printf("Free     : %ld MB\n", free / 1024);
printf("Available : %ld MB\n", available / 1024);
printf("\n");
}

```

```
// Fungsi untuk membaca disk usage
```

```
void get_disk_info() {
```

```
    printf("DISK USAGE\n");
```

```
    print_line(50);
```

```
    struct statvfs buf;
```

```
    if(statvfs("/", &buf) != 0) {
```

```
        printf("Error: Cannot get disk info\n");
```

```
        return;
```

```
    }
```

```
    unsigned long total = buf.f_blocks * buf.f_frsize;
```

```
    unsigned long free = buf.f_bfree * buf.f_frsize;
```

```
    unsigned long used = total - free;
```

```
    float percent = (float)used / total * 100;
```

```
    printf("Total    : %.2f GB\n", total / (1024.0 * 1024 * 1024));
```

```
    printf("Used      : %.2f GB (%.1f%%)\n", used / (1024.0 * 1024 * 1024), percent);
```

```
    printf("Free       : %.2f GB\n", free / (1024.0 * 1024 * 1024));
```

```
    printf("\n");
```

```
}
```

```
// Fungsi untuk membaca running processes
```

```
void get_process_info() {
```

```
    printf("RUNNING PROCESSES\n");
```

```
    print_line(50);
```

```
    DIR *dir = opendir("/proc");
```

```
    if(!dir) {
```

```
        printf("Error: Cannot open /proc\n");
```



```

    return;
}

printf("%-8s %-20s %s\n", "PID", "NAME", "STATE");
print_line(40);

struct dirent *entry;
int count = 0;

while((entry = readdir(dir)) != NULL && count < 20) { // Batasi 20 proses
    if(entry->d_type == DT_DIR && atoi(entry->d_name) > 0) {
        char path[256];
        snprintf(path, sizeof(path), "/proc/%s/comm", entry->d_name);

        FILE *file = fopen(path, "r");
        if(file) {
            char name[100];
            if(fgets(name, sizeof(name), file)) {
                name[strcspn(name, "\n")] = 0;

                // Baca status
                snprintf(path, sizeof(path), "/proc/%s/status", entry->d_name);
                FILE *status_file = fopen(path, "r");
                char state[20] = "Unknown";
                if(status_file) {
                    char status_line[100];
                    while(fgets(status_line, sizeof(status_line), status_file)) {
                        if(strstr(status_line, "State:")) {
                            sscanf(status_line, "State:\t%s", state);
                            break;
                        }
                    }
                }
            }
        }
    }
    count++;
}

```

```

        }
    }
    fclose(status_file);
}

printf("%-8s %-20s %s\n", entry->d_name, name, state);
count++;
}
fclose(file);
}
}
}
closedir(dir);
printf("\n");
}

```

// Fungsi untuk membaca network interfaces

```

void get_network_info() {
    printf("NETWORK INTERFACES\n");
    print_line(50);

    FILE *file = fopen("/proc/net/dev", "r");
    if(!file) {
        printf("Error: Cannot open /proc/net/dev\n");
        return;
    }
}

```

```

char line[256];
int line_count = 0;

```

```

printf("%-12s %-12s %s\n", "INTERFACE", "RX Bytes", "TX Bytes");
print_line(40);

while(fgets(line, sizeof(line), file)) {
    if(line_count++ < 2) continue; // Lewati 2 baris header

    char interface[20];
    unsigned long rx_bytes, tx_bytes;

    if(sscanf(line, "%[^:]: %lu %*u %*u %*u %*u %*u %*u %*u %*u %lu",
        interface, &rx_bytes, &tx_bytes) == 3) {
        // Hapus spasi di nama interface
        char *iface = interface;
        while(*iface == ' ') iface++;

        printf("%-12s %-12lu %lu\n", iface, rx_bytes, tx_bytes);
    }
}
fclose(file);
printf("\n");
}

// Fungsi untuk load average (NIM genap)
void get_load_avg() {
    printf("LOAD AVERAGE\n");
    print_line(50);

    FILE *file = fopen("/proc/loadavg", "r");
    if(!file) {
        printf("Error: Cannot open /proc/loadavg\n");
    }
}

```

```

        return;
    }

    char line[100];
    if(fgets(line, sizeof(line), file)) {
        float load1, load5, load15;
        sscanf(line, "%f %f %f", &load1, &load5, &load15);
        printf("1 minute : %.2f\n", load1);
        printf("5 minutes : %.2f\n", load5);
        printf("15 minutes: %.2f\n", load15);
    }
    fclose(file);
    printf("\n");
}

int main() {
    clear_screen();
    printf("=== SYSTEM INFORMATION TOOL ===\n\n");

    get_cpu_info();
    get_memory_info();
    get_disk_info();
    get_process_info();
    get_network_info();
    get_load_avg(); // Untuk NIM genap

    return 0;
}

```

BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan pelaksanaan praktikum sistem operasi, dapat disimpulkan bahwa seluruh program berhasil mengimplementasikan konsep inti sistem operasi dengan efektif. Program Process Manager mendemonstrasikan kemampuan multiprocessing dan IPC, File Monitor membuktikan kinerja real-time monitoring yang andal, Custom Shell berhasil menangani fitur advanced seperti multiple pipes dan background processes, sementara System Information Tool mampu mengekstrak data sistem secara akurat. Seluruh program menunjukkan optimasi yang baik dengan zero memory leaks dan penanganan signal yang proper, membuktikan pemahaman mendalam tentang mekanisme internal sistem operasi Linux melalui pengembangan aplikasi tingkat sistem yang langsung berinteraksi dengan kernel.